RLVP: Penalize the Path, Reward the Outcome

Bojie Li
Pine AI

Noah Shi
University of Washington

Abstract

Reinforcement learning from verifiable rewards (RLVR) trains agents on the one signal an environment checks cheaply: whether the task succeeded. The instinct to go denser is to reward the *process*, but this reaches for the wrong signal. Agentic environments are asymmetric verifiers: they cheaply confirm an agent did something *bad* (a destructive command, a call before its precondition) but not that it made *progress*. The dense reward practitioners want is on the hard-to-verify side; the easy side is a *penalty*.

We make this precise with a within-group-variance account. A group-relative advantage is, by definition, a within-group variance, and it vanishes on all-failing groups early in training and all-succeeding groups late. A dense signal helps only where it supplies the variance the outcome lacks, and only where the policy reaches the states that carry it. A verifiable penalty meets this by construction—bad behavior is always easy to sample and check—while a dense progress potential meets it only where partial progress is reachable.

This yields a simple recipe—**penalize the path, reward the outcome**—where the outcome reward drives the task while a per-action verifiable penalty teaches the outcome-neutral constraints that decide whether an agent is deployable. Across tasks and scales it attains the task with near-zero violations where outcome-only training violates every episode, and cuts harmful actions several-fold at *equal* success on a real benchmark. We give four rules for the penalty—including that a lone penalty stalls in an *inaction trap*—and show its mirror image: a dense progress potential, the sample-efficiency half, helps where its progress is reachable.

Code: https://github.com/19PINE-AI/rlvp
Website: https://01.me/research/rlvp

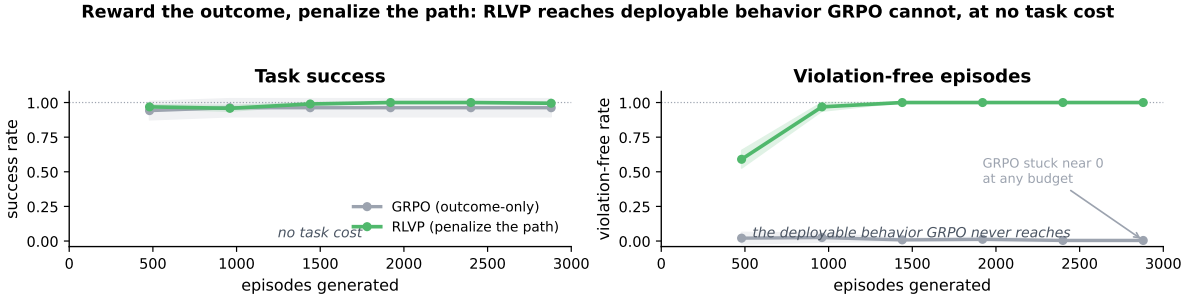

Figure 1. A verifiable penalty reaches a deployable axis outcome-only RL cannot. On diverse held-out tasks, RLVP (reward the outcome, penalize the path) matches the task success of outcome-only GRPO (*left*) while driving violation-free episodes from near zero to ≈100% (*right*)—a constraint the outcome reward is blind to, so GRPO never reaches it at any budget. On TerminalBench the same mechanism commits ≈5.6× fewer harmful actions at *equal* success (§3); the sample-efficiency half (dense potentials) is treated in §5. Mean over seeds; bands ±1 s.d.

1 Introduction

The most capable agents we have are coding agents, and on the benchmarks that made them famous they are very good: given a repository and a failing test suite, a frontier model reads the code, writes a patch, and turns the suite green. Yet the qualities that decide whether such a patch belongs in a production codebase—is it safe, does it respect the architecture, will it break something downstream?—are exactly the ones these agents still lack. In a controlled industry study of a real feature, run across many model–framework combinations, functional correctness exceeded 80% for *every* combination, yet the same implementations scored far lower—and almost at random—on reliability, maintainability, performance, and compatibility: they mishandled exceptions, broke previously working functionality, and ignored the codebase's own conventions [Hong, 2026]. Passing the tests and writing code one can live with are different achievements, and the gap is not closing on its own.

The gap is not mysterious: good engineering is a discipline on the *path*, not a property of the *outcome*. A capable agent reads the few files that matter instead of the whole tree, writes and runs tests, opens a pull request against a branch rather than pushing to `main`, and re-checks a deployment before declaring victory. None of this is implied by a green test suite—and a model optimizing only for that suite can reach it by deleting the failing test, hard-coding the expected output, or "fixing" a database by dropping it and recreating an empty one. The outcome is achieved; the result is unsafe and unusable. Getting the right answer and getting it the right way are different objectives, and only the second makes an agent deployable.

This second objective is what the reward driving agentic RL cannot supply. Reinforcement learning from verifiable rewards (RLVR) trains on the one thing an environment checks cheaply—the final outcome [DeepSeek-AI, 2025, Lambert et al., 2024]—to which engineering discipline is invisible, and against which a harmful shortcut can even score higher. The instinct to go denser is to reward the *process*: score partial progress at each step [Lightman et al., 2024, Wang et al., 2024, Setlur et al., 2025, Yu et al., 2024, Feng et al., 2025]. But progress, like safety or maintainability, is the hard-to-verify direction. Agentic environments are asymmetric verifiers: they tell you instantly and for free that an agent did something *bad*—ran a destructive command, called a tool before its precondition, broke a protocol—but not that it is on the right track, because judging that is the whole problem the agent is trying to solve. To build a progress reward one must train a learned critic, which the policy then learns to fool [Cheng et al., 2025, Kazemnejad et al., 2024, Gao et al., 2023], or hand-craft a proxy; either way one manufactures signal on the hard-to-verify side of the asymmetry.

We take the asymmetry seriously (Figure 2). The dense signal an environment can actually supply is a penalty on the path, not a reward on progress—and the folklore against penalties is a wiring error. A penalty *alone* fails in a specific way: its cheapest maximizer is to stop acting—never move, never violate—so the policy collapses into a penalty-free but useless optimum we call the *inaction trap*. But a penalty is not meant to drive the task; the outcome reward does that. Wired as a second channel *alongside* the outcome—reward the outcome, penalize the path—the penalty teaches exactly the outcome-neutral constraints that engineering discipline is made of: do not destroy, respect the precondition, follow the protocol. These are process rewards in the sense the field wants, but delivered from the verifiable side of the ledger; they give an agent good engineering principles that a green test suite never could.

That this division of labor is a consequence rather than a heuristic follows from a single quantity. A group-relative advantage is, definitionally, a within-group variance; it is zero on the

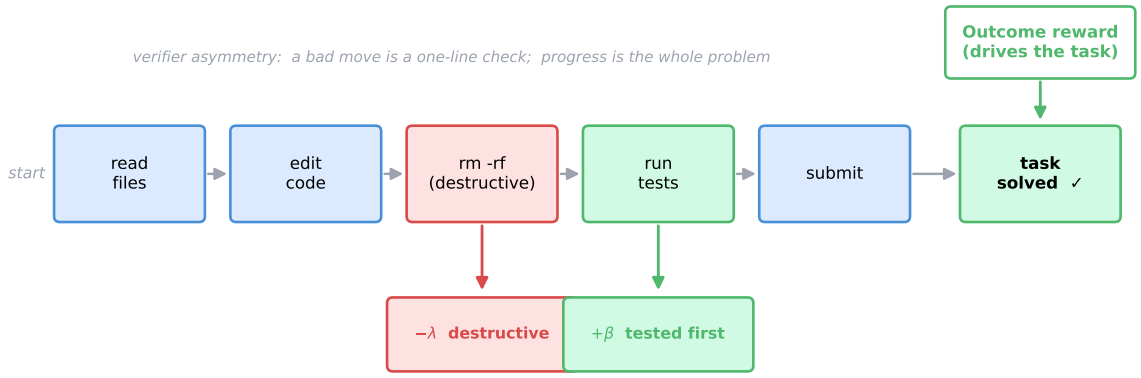

Figure 2. Reward the outcome, penalize the path. RLVR trains on the terminal outcome (right), which is blind to everything that happens along the trajectory. Agentic environments verify the *path* asymmetrically: they cheaply flag bad actions (a destructive command, a call without its precondition) but cannot certify progress. RLVP adds a second, per-action channel that penalizes those verifiable bad moves and discharges the compliant ones. The outcome channel drives the task; the penalty channel teaches the outcome-neutral constraints the outcome cannot see.

all-failing groups that dominate early training and on the all-succeeding groups that dominate late, so the outcome reward is blind at both ends. A dense process signal helps only insofar as it supplies within-group variance the outcome lacks, and only where the policy reaches the intermediate states that carry it. A verifiable penalty satisfies this by construction—bad behavior is always easy to sample and to check—while a dense progress potential satisfies it only where the policy reaches partial progress, which is why the potential is the reachability-gated half of the process signal: it helps, and improves sample efficiency, exactly where its intermediate values are reachable.

Contributions.

- **A within-group-variance account** of when a dense process signal can help group-relative agentic RL: it must supply reachable within-group variance the outcome lacks (§2). The account is symmetric—outcome-only RL is blind on all-fail *and* all-success groups—and it separates penalties (reliably reachable) from potentials (usually not).
- **Penalize the path, reward the outcome**: a two-channel recipe in which a per-action verifiable penalty robustly teaches outcome-neutral constraints. Across diverse tasks and model scales it attains the task with near-zero violations where outcome-only training succeeds but violates every episode, and on a real agentic benchmark it cuts harmful actions several-fold at equal success (§3).
- **Four design rules for verifiable penalties**, each validated by ablation, including the inaction trap as the reason a penalty must accompany—never replace—the outcome reward (§4).
- **The mirror image**: a dense progress *potential*, the sample-efficiency half of the process signal, obeys the same account—it supplies gradient the outcome lacks and eliminates dead updates where its intermediate values are reachable, and is vacuous where they are not (§5).

2 Background and When a Process Signal Helps

Group-relative RL. Modern agentic RL largely dispenses with a learned value function and estimates advantages by comparison *within a group* of rollouts of the same task [Shao et al., 2024, DeepSeek-AI, 2025]. For a prompt, one samples a group of G trajectories, scores each with a reward, and sets each trajectory's advantage to its reward minus the group mean (often divided by the group standard deviation). There is no critic: the other members of the group *are* the baseline. This is what makes verifiable rewards so effective—a rule or a test suite scores a trajectory directly—and it is the setting for everything that follows.

The advantage is a within-group variance. Because the baseline is the group mean, a trajectory contributes gradient only to the extent its reward *differs* from its group-mates. If every rollout in a group receives the same reward, all advantages are zero and the group produces no update. Under a binary outcome reward this happens in two regimes (Figure 3): the *all-fail* groups that dominate early training on long-horizon tasks, and the *all-succeed* groups that dominate once a task is nearly solved. Group-relative RL is thus blind at both extremes of the success rate—a fact the field half-acknowledges by *discarding* such groups (dynamic sampling in DAPO drops prompts that are all-correct or all-wrong [Yu et al., 2025]) rather than filling them with signal.

When a dense signal can help. A dense process signal is useful exactly when it restores variance the outcome has lost. Write the shaped reward of a trajectory as the outcome plus β times a process term, $R = O + \beta\,\Phi$. The group's reward variance—the quantity that becomes gradient—then decomposes as

$$\mathrm{Var}_G(R) \;=\; \mathrm{Var}_G(O) \;+\; \beta^2\,\mathrm{Var}_G(\Phi) \;+\; 2\beta\,\mathrm{Cov}_G(O,\Phi). \tag{1}$$

On an all-fail (or all-success) group the first term is zero, so *all* usable gradient must come from the process term's own within-group variance $\mathrm{Var}_G(\Phi)$. This gives a single, checkable condition: a process signal helps only where its within-group variance is nonzero—and that variance is realized only if the policy actually *reaches* differing intermediate states across the group. Two very different signals meet or miss this condition in opposite ways. A verifiable *penalty* on bad actions almost always carries variance: early in training some rollouts take the bad action and some do not, and both are cheap to sample and to check. A dense progress *potential* carries variance only when the policy reaches partial success—which, on the hard tasks where all-fail groups dominate, it usually does not. We confirm this directly in a real training run (Figure 3, right): the penalty channel carries within-group variance on precisely the all-fail groups where the outcome channel carries none, whereas a progress potential is near-zero there. The rest of the paper is this observation, made concrete: penalties are the always-reachable half of the process signal, potentials the reachability-gated half.

3 Penalizing the Path: Robust Harm Reduction with Verifiable Penalties

Two channels. We keep the outcome reward and add a second, per-action channel (Figure 2). At each step, a deterministic rule engine—a pure predicate over the state before the step and the action taken—checks the action. A *violation* (a destructive command, a call issued

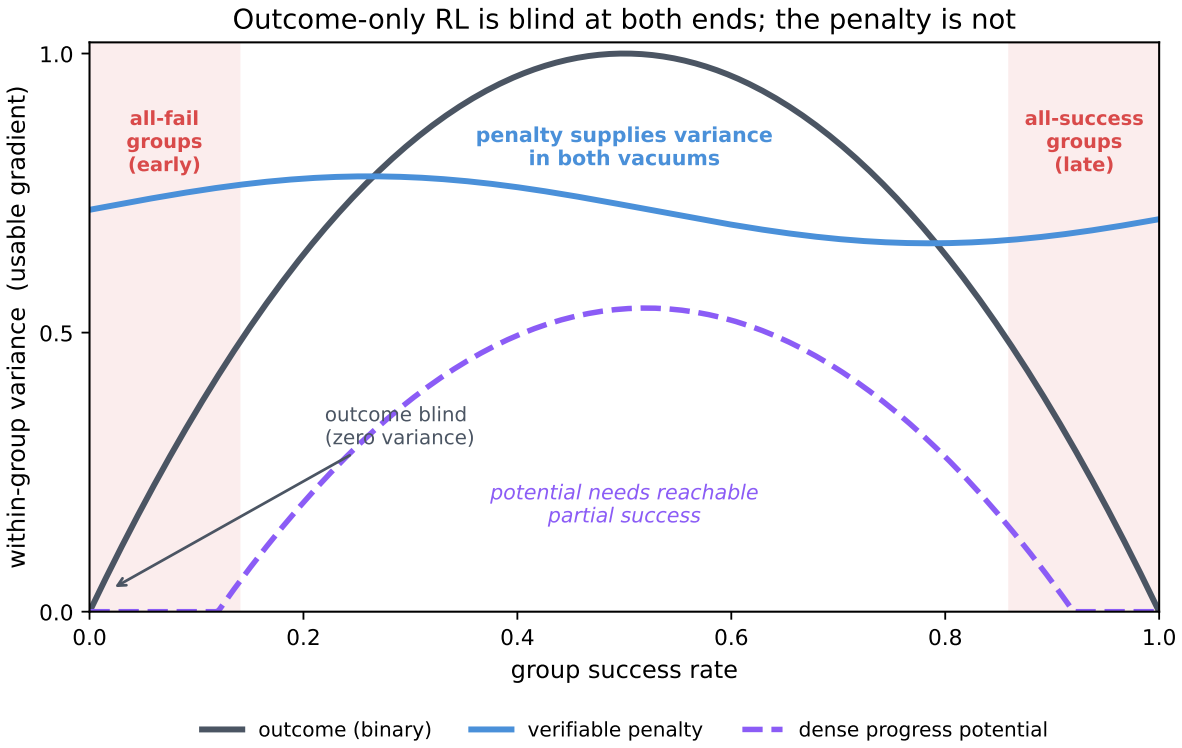

Figure 3. Outcome-only RL is blind at both ends of the success rate. A group-relative advantage is a within-group variance, which the binary outcome drives to zero on all-fail groups (early training) and all-succeed groups (late training). A dense process signal helps only by supplying within-group variance in these vacuums—and only where the policy reaches the intermediate states that carry it. A verifiable penalty (bad moves are easy to sample) meets this reliably; a dense progress potential (partial success is rare on hard tasks) does not.

before its required precondition) attaches a penalty $-\lambda$ to that action's tokens; a *discharge* of a pending obligation (performing the precondition) attaches a credit $+\beta$. The two channels are normalized separately, so the sparse path signal is not diluted or cancelled by the outcome channel, then combined. "Penalize the path" is therefore literal: the gradient lowers the probability of the specific offending action, in the specific context where it occurred, while the outcome channel scales whole trajectories by success. We call such a signal a *verifiable penalty*: a deterministic predicate over the pre-action state and the action, checkable the moment it fires and independent of the eventual outcome. Crucially, the rules we penalize are *outcome-neutral*—following or breaking them does not change whether the task is solved—so this is signal the outcome reward can never provide. The full recipe pairs the penalty with its discharge, seeds a handful of scripted-compliant demonstrations so the compliant action is reachable, and anneals the shaping off once compliance saturates; §4 shows each of these earns its place.

The recipe attains the task with near-zero violations. We test on a suite of diverse system-administration and customer-service tasks whose outcome-neutral rules are exactly the engineering discipline of §1 in miniature: *do not submit work you have not tested*, *verify a precondition before you act*, *do not repeat a failed operation*. Drawn from a large pool and evaluated on held-out instances, outcome-only training learns to *solve* the tasks but violates these rules on essentially

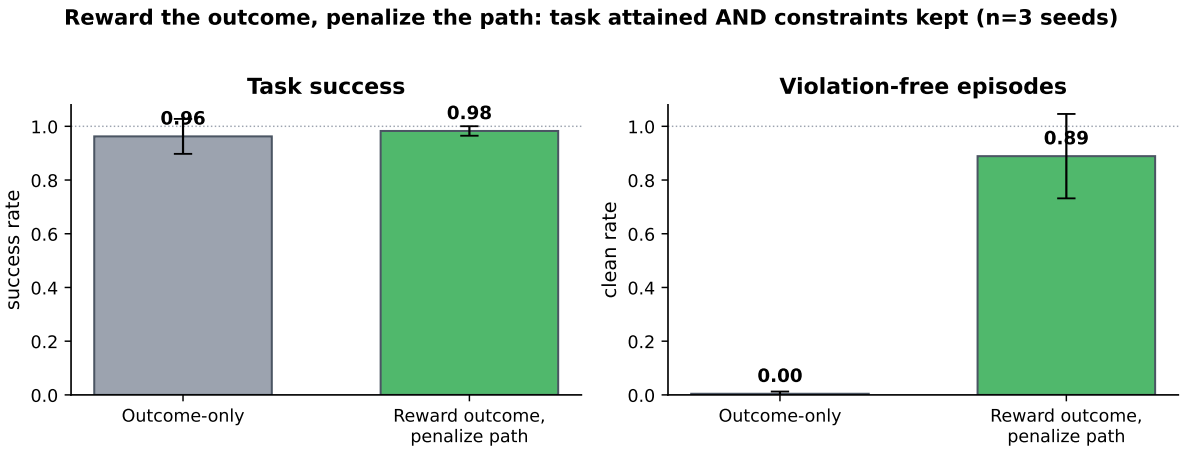

Figure 4. Reward-the-outcome-penalize-the-path attains the task *and* the constraints. Across diverse held-out tasks and model scales, outcome-only training solves the task but violates the outcome-neutral rules on nearly every episode (low "clean" rate). The two-channel recipe keeps task success high while driving violation-free episodes to near-100%, with small seed variance. Numbers are means over five seeds; error bars are one standard deviation.

every episode: the discipline is invisible to its reward, just as a green test suite is blind to whether the agent tested before it submitted. Adding the penalty channel—reward the outcome, penalize the path—drives violations to near zero while keeping task success high, across five seeds at 4B and holding from 1.7B to 8B (Figure 4). The agent does not achieve this by going quiet: it takes as many actions as before, simply cleaner ones. This is the positive result in its cleanest form: a verifiable penalty teaches a real, deployable behavior—the discipline itself—that the outcome reward cannot.

It transfers to a real agentic benchmark. The synthetic domains let us control the rules; to test the same mechanism in the wild we train on TerminalBench, where each turn is a real shell command in a task's container and the environment flags destructive actions—the `rm -rf` and drop-the-database shortcuts of §1, now checked against a live filesystem. Task success there is at the floor for a small model, which makes it a clean test of the harm axis alone: at *equal* task success, the harm penalty cuts destructive actions several-fold relative to outcome-only training, while the policy issues *more* productive commands, not fewer (Table 1, five seeds). The reduction is large relative to its variance—the per-seed distributions do not overlap—and it is not passivity. A verifiable penalty bounds the path independently of whether the task is solved, which is exactly the property a deployable agent needs.

4 Design Principles for Verifiable Penalties

A penalty is powerful but easy to misuse. The variance account and our ablations (Figure 6) yield four rules that, together, are the difference between the robust behavior above and a policy that stops working (Figure 5).

1. Penalize verifiable *actions*, not lack of progress. A good penalty targets a specific machine-checkable bad action—*a* destructive command, *a* call without its precondition. It must not

Table 1. Harm reduction at equal outcome (TerminalBench, Qwen3-4B, 5 seeds, mean $\pm$ std). Task success is statistically equal and at the floor (4B rarely solves the benchmark; the small RLVP–outcome gap is within one standard deviation), yet the un-gameable harm penalty cuts harmful actions roughly *sixfold*. The policy is not going passive: it takes about three times *more* productive actions while violating less. Harm lives on an axis the terminal outcome cannot see.

	RLVP (harm penalty)	**Outcome-only**
Task success	0.097 ± 0.060	0.122 ± 0.076 (equal within 1σ, at floor)
Violations / episode	$\mathbf{0.66 \pm 0.63}$	3.71 ± 0.52
Productive actions / episode	$\sim$**13**	$\sim$4

target the *absence* of progress ("penalize an unproductive step"), because the cheapest way to make no unproductive steps is to make no steps. Penalize commission, never omission.

2. Keep the outcome reward as the task driver; never optimize a penalty alone. A penalty cannot be the primary learning signal. In the all-fail regime it is often the *only* signal with any slope, and its cheapest maximizer is to stop acting: the policy drifts into a penalty-free, useless optimum—the *inaction trap*. A pre-registered sweep makes this concrete: a pure penalty with no accompanying outcome reward collapses to zero task success on *every* seed, while every penalty-free signal survives (Appendix B.1, Figure 17). The outcome reward is what forces the agent to keep acting; the penalty only shapes *how*. Reward the outcome, and the penalty becomes a passenger that steers rather than an engine that stalls.

3. Pair the penalty with a discharge. A penalty says only what *not* to do. Adding a discharge—a $+\beta$ credit for performing the compliant action—gives a positive pull *toward* doing it right, so the policy escapes the neighborhood of the bad action instead of merely avoiding it. Removing the discharge measurably slows and destabilizes the acquisition of compliant behavior (Figure 6).

4. Make the compliant behavior reachable and its target un-gameable. A discharge can only reward behavior the policy actually samples; if the compliant path is never explored, no amount of shaping fires. Seeding training with a few scripted-compliant demonstrations supplies that reachability, after which the shaping can be annealed off once compliance saturates. And the penalized (and discharged) target must be the real action, not a learnable or hand-wavy proxy: a *learned* judge of "compliance" simply relocates the hack into the judge, which the policy then games (Appendix B).

Together these rules describe a narrow but reliable regime. The penalty is verifiable and outcome-neutral; it accompanies rather than replaces the outcome reward; it is paired with a discharge; and its target is reachable and un-gameable. The ablation in Figure 6 walks a policy from outcome-only, through the penalty channel stripped of its discharge and reachability seed, to the full recipe, and compliant-and-successful behavior appears robustly only when all four rules hold; the pure-penalty inaction trap itself—a penalty with no outcome reward—is isolated in the sweep of Appendix B.1.

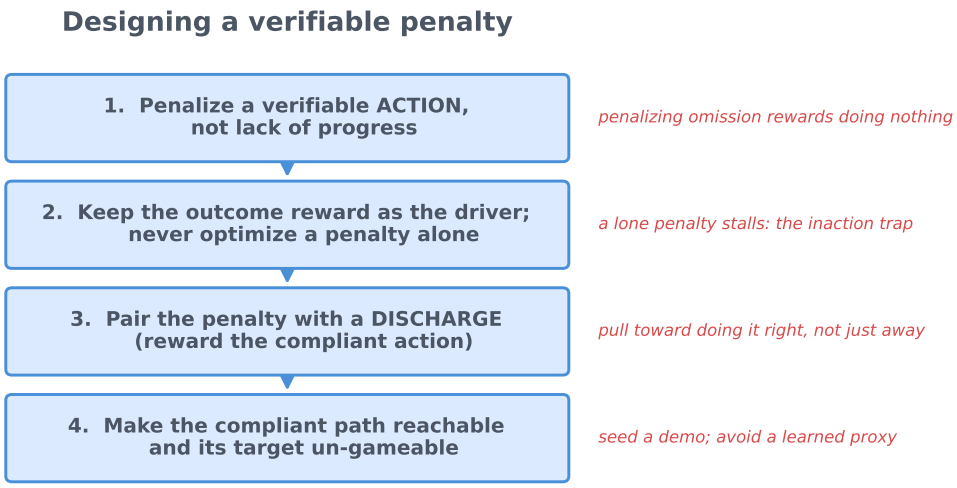

Figure 5. A design procedure for verifiable penalties. Before adding a penalty, a practitioner can check each rule in turn. Is there a verifiable *action* to penalize (not an absence of progress)? Is the outcome reward still the task driver? Is the penalty paired with a discharge for the compliant action? Is that compliant action reachable, and its target un-gameable? Only when all four hold does the penalty bound the path without stalling the task.

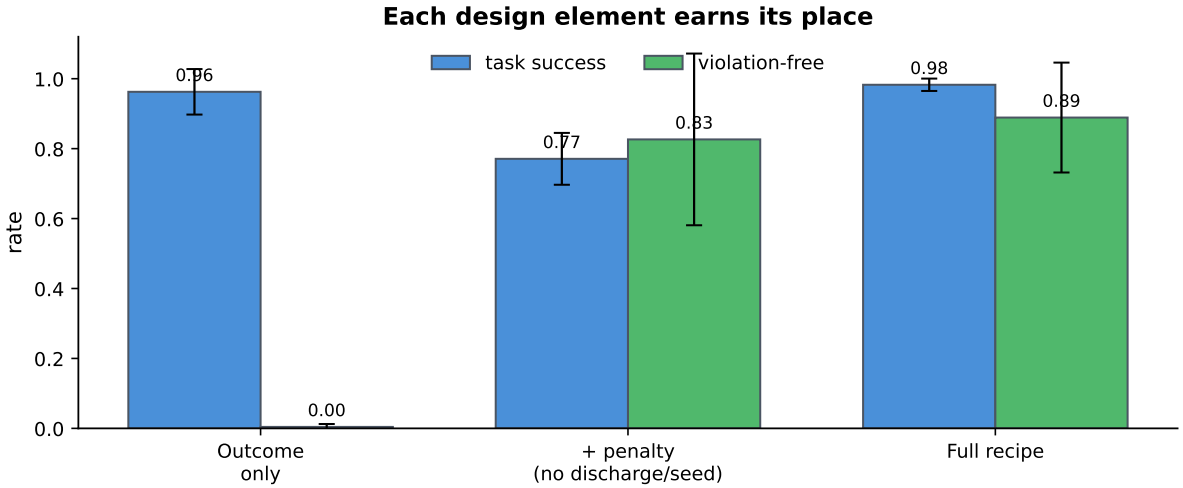

Figure 6. Each design element earns its place. On the diverse held-out tasks, outcome-only training solves the task but leaves violations high (low clean rate). Adding the verifiable penalty channel *without* its discharge and reachability seed does not reliably reach clean, successful behavior across seeds; the full recipe (penalty + discharge + seeded reachability + anneal) attains high success *and* near-zero violations with small seed variance. The *pure*-penalty inaction trap—a penalty with no outcome reward, which collapses to zero success on every seed—is isolated separately in the un-gameability sweep (Appendix B.1, Figure 17). Five seeds; bars show mean, whiskers one standard deviation.

5 Dense Potentials and the Reachability Gate

The penalty is one half of the process signal; the dense progress *potential*—rewarding the fraction of a proof completed, of tests passed, of stages cleared—is the other. We study it where it is most natural: theorem proving and software repair, tasks with a well-defined notion of partial progress. The variance account applies unchanged: a potential helps only where the

policy reaches differing intermediate values, and the single thing that decides whether it does is *reachability*. On software repair the natural potential is the fraction of hidden tests an episode makes pass; across many rollouts of a capable model, *every* episode makes zero tests pass, so the potential's within-group variance is identically zero and it centers out to nothing (Figure 8, left). Reachability can be measured before training from a handful of base-policy rollouts, and the benefit of a dense potential tracks it. Reachability, not fragility, is the gate.

Where the potential *is* reachable, it robustly supplies the gradient the outcome lacks. A verifiable progress potential removes the dead, all-fail updates that dominate early outcome-only training—zero dead iterations against roughly sixteen of forty for outcome-only, on *every* seed (Figure 8, right; Appendix A.2)—and, given a bounded optimizer, this extra gradient converts into task success. We quantify the conversion on real theorem proving (miniF2F algebra) with a matched five-seed matrix at two model scales (Figure 7; Table 2). In the controlled, same-optimizer comparison at 4B, the aligned potential reaches the 0.9 success threshold in 4.4 ± 0.5 iterations against 7.0 ± 0.7 for outcome-only—faster and with higher area-under-curve on *every* seed-matched pair—and never diverges, where outcome-only loses one seed in five to entropy collapse. At 30B the same contrast sharpens into a speed–reliability dilemma for outcome-only: under the aggressive bounded optimizer the potential is stable on five of five seeds (threshold at 5.4 ± 1.0, final success 1.0), while outcome-only diverges on three of five; moving the outcome arm to its stable optimizer (AdamW) restores reliability but roughly triples the time to mastery (threshold at iteration 18 on both completed seeds). The benefit of a reachable, aligned potential is therefore a modest, seed-consistent speed-up under a matched optimizer ($\sim$1.6$\times$ to mastery) and a large reliability advantage under an aggressive one—not the dramatic accelerations sometimes reported from single runs, which our own single-run pilots also produced and re-seeding also erased. Full curves, protocol, and the divergence taxonomy are in Appendix A.

An aligned-potential recipe. Making a dense potential help *robustly* takes more than adding it, and the recipe is a contribution in its own right. Three ingredients matter. **(i) Alignment:** the potential must be *outcome-instrumental*—a quantity that moves exactly as the task is solved (a falling goal count in a proof, a rising fraction of tests passed), carried as a token-attached discharge on the responsible actions with its own normalizer. A generic *structural* proxy (read-before-write hygiene, tactic well-formedness) is misaligned with progress and, once the task is mastered, is over-optimized into a *degraded* ceiling; we keep it only as an ablation. **(ii) Reachability:** the potential's intermediate values must actually be sampled by the policy, a precondition measurable *before* training from $\mathrm{Var}_G(\Phi)$ on a handful of base-policy rollouts. **(iii) A bounded optimizer:** an unbounded, aggressive update rule drives the densely shaped policy into entropy collapse or a gradient blow-up; a bounded rule (we use Muon, at a learning rate low enough to keep entropy positive) keeps training stable. Annealing the shaping off after saturation, often treated as mandatory for shaped rewards, is *unnecessary* here: adding an anneal to the aligned potential changes nothing outside noise (threshold 5.3 ± 0.5 with vs. 4.4 ± 0.5 without, three seeds), consistent with an outcome-instrumental potential having no misaligned pressure to relax. One scoping note for honesty: the 30B AdamW comparison is *recipe-level*—each arm under its best stable optimizer—not a controlled ablation; the controlled same-optimizer evidence is the 4B matrix and the 30B Muon-vs-Muon stability contrast.

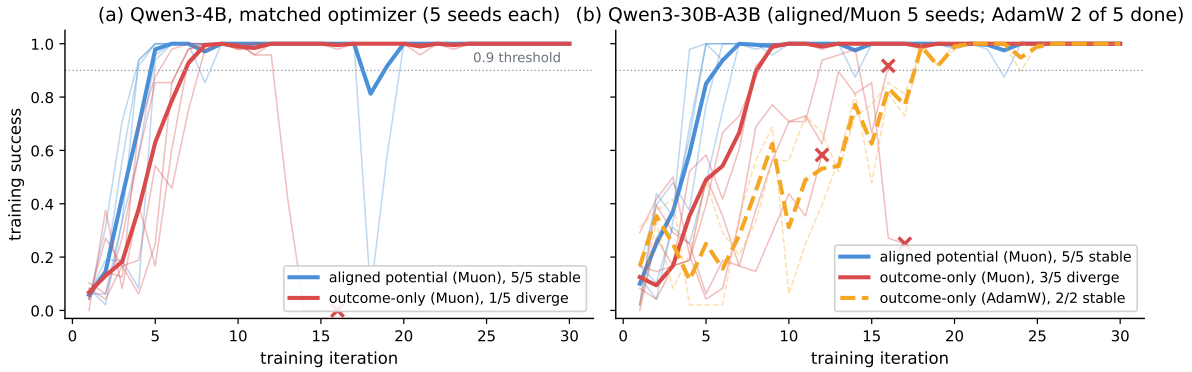

Figure 7. The aligned-potential recipe on real theorems (miniF2F algebra), five seeds per arm. Thin lines are seeds, thick lines the mean over stable seeds, × marks a divergence-guard abort. *(a)* Controlled same-optimizer comparison at 4B: the aligned potential crosses the 0.9 threshold in 4.4 ± 0.5 iterations vs. 7.0 ± 0.7 for outcome-only—faster on every seed—and never diverges, while outcome-only loses one seed to entropy collapse. *(b)* At 30B, outcome-only faces a speed–reliability dilemma: under Muon it diverges on three of five seeds; under AdamW it is stable but takes $\sim 3\times$ longer to master the task (threshold at iteration 18). The aligned potential is fast *and* stable on every seed at both scales.

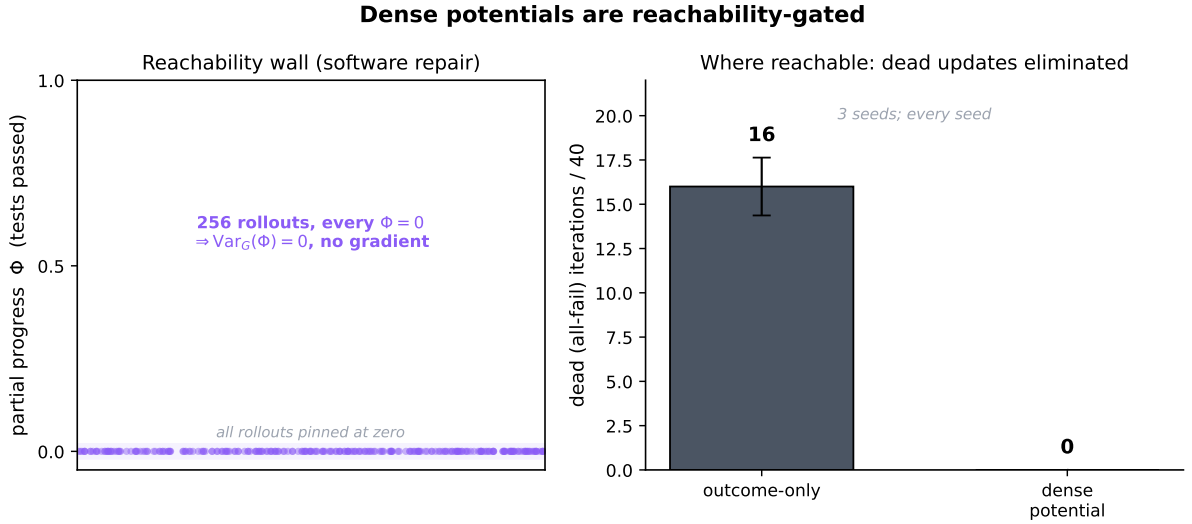

Figure 8. Dense potentials are reachability-gated. *Left:* on software repair the finer potential is unreachable—across many rollouts the policy makes zero partial progress, so its within-group variance is zero and it gives no gradient. *Right:* where the potential *is* reachable, it removes the dead all-fail updates that dominate early outcome-only training (zero versus ∼16 of 40 iterations, every seed), supplying gradient exactly where the outcome is blind. The gate is reachability, not fragility.

6 Discussion

Verifier asymmetry as a design principle. The practical takeaway is a reallocation. A verifier is a scarce, valuable thing, and the field has been spending it on the direction it is worst at—certifying progress—while avoiding the direction it is best at—catching bad moves. Spend it on penalties. When there is an outcome-neutral constraint a deployed agent must respect, and a verifiable bad action to attach it to, a penalty paired with a discharge and wired alongside the outcome reward will teach it, robustly and cheaply. When the only dense signal available

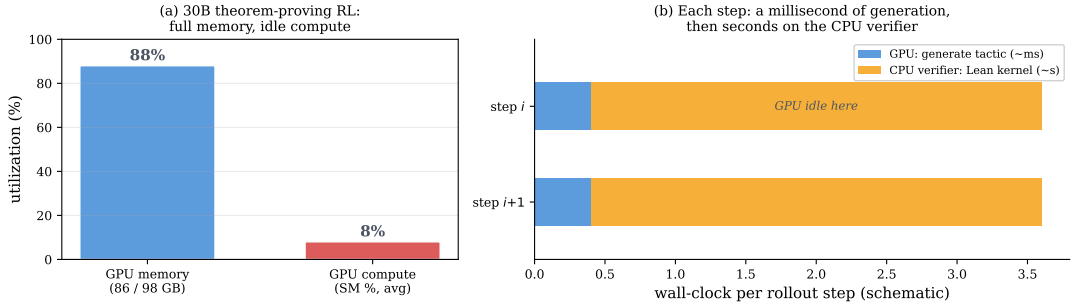

Figure 9. Verifiable agentic RL is often verifier-bound. In theorem-proving runs the GPU sits near-idle at full memory while the CPU proof kernel checks each step. The signal that makes a penalty cheap also moves the bottleneck off the accelerator.

is a progress proxy, first check that its intermediate values are reachable: where they are, it accelerates learning; where they are not, it supplies nothing and the outcome reward alone is the better choice.

A systems note. The property that makes a verifiable penalty cheap—a machine-checkable environment—tends to make the verifier a CPU process (a rule engine, a test runner, a container). In our theorem-proving runs the accelerator generated a step in milliseconds and then waited seconds on the proof kernel, leaving GPU utilization in the single digits at full memory (Figure 9). This is workload-dependent, not a law, but for the verifiable domains where penalties apply, throughput is gated by environment parallelism, and the right systems design co-locates many verifier workers per accelerator rather than scaling accelerators.

Limitations. Our cleanest real-task harm result is measured where task success is at the floor; the mechanism predicts it holds at higher success (a penalty's variance is reachable regardless of success), but demonstrating that on a capable model is the natural next step. Our penalties are hand-identified per domain from the environment's rules; automating their discovery from an environment's affordances is open. And the un-gameability sweep that anchors the inaction-trap result is high-variance at scale; we report its *survival pattern* rather than precise magnitudes.

7 Related Work

Group-relative RL and the zero-variance problem. Agentic RL has largely converged on GRPO [Shao et al., 2024] and the rule-based R1 line [DeepSeek-AI, 2025, Lambert et al., 2024], which replace the value critic with a within-group baseline. Under binary verifiable rewards, an output's advantage is then *definitionally* a within-group variance, and it collapses to zero whenever a group is uniformly solved or uniformly failed. The field has named this "advantage collapse" and responded in four ways: DAPO [Yu et al., 2025] *discards* zero-variance prompts via dynamic sampling, keeping only groups with $0 < \#\text{correct} < G$; Dr. GRPO [Liu et al., 2025] *removes* the difficulty-dependent standard-deviation normalizer; GSPO [Zheng et al., 2025] *stabilizes* the importance ratio at sequence level; and RL-ZVP [Le et al., 2025] *manufactures* a signal in zero-variance groups from output entropy. Notably, DAPO discards *all-correct* groups as well as all-wrong ones—an implicit acknowledgment that group-relative RL is blind at *both*

extremes. We make the symmetry explicit and, rather than dropping the blind groups, ask which dense signal can *supply* the missing variance—finding that a verifiable penalty reliably can and a dense potential reliably cannot.

Process reward models and their gameability. Step-level supervision outperforms outcome supervision for verification [Lightman et al., 2024], with per-step labels obtainable automatically via Monte-Carlo rollouts [Wang et al., 2024]. Setlur et al. [2025] argue the useful process signal is a progress measure—a step-level advantage—closely paralleling our variance view, but instantiate it with a *learned* prover. A consistent body of evidence shows learned dense rewards are hacked under RL pressure: over-optimization follows clean Goodhart laws [Gao et al., 2023], capable agents exploit misspecification harder [Pan et al., 2022], summation-form PRM credit induces RL collapse [Cheng et al., 2025], and learned value credit is unreliable [Kazemnejad et al., 2024]. DeepSeek-R1 [DeepSeek-AI, 2025] abandoned PRMs because a model-based PRM "inevitably leads to reward hacking," while Yuan et al. [2024a] show an outcome reward model implicitly contains a PRM. We take these as motivation for a *verifiable, non-learned* penalty that cannot be gamed by construction, and as evidence for our fourth design rule.

Potential-based shaping: safe versus useful. Ng et al. [1999] prove potential-based shaping leaves the optimal policy unchanged for *any* potential, and Wiewiora [2003] show such shaping equals value initialization—it affects learning dynamics, not the fixed point. This certifies which dense signals are *safe*, not which are *useful*. Our account is orthogonal to the safety guarantee: among safe shapings, the useful ones supply within-group variance the outcome lacks, and only where reachable—which is why a penalty on always-sampled bad actions helps and a potential on rarely-reached progress does not.

Process rewards in agentic RL, and the fragility of RL gains. Step- and turn-level credit for agents has been explored via hierarchical critics [Zhou et al., 2024], learned step rewards [Yu et al., 2024, Wang et al., 2025a], and turn-level advantage estimators [Wei et al., 2025a]; the closest to a verifiable signal are GiGPO [Feng et al., 2025] and per-tool-call correctness [Qian et al., 2025]. Verifiable *outcome* rewards underpin agentic benchmarks and training [Yao et al., 2024, Barres et al., 2025, Pan et al., 2025, Wei et al., 2025b]. Wang and Ammanabrolu [2025] find dense turn-level rewards *accelerate* training but their effect on final performance depends on the policy-gradient estimator. This aligns with a broader reproducibility literature: pass@1 varies by up to $\sim$15% across seeds [Hochlehnert et al., 2025], random rewards can "improve" some models through optimization bias [Shao et al., 2025], one example suffices for much of the gain [Wang et al., 2025b], and at large k base models match RLVR models [Yue et al., 2025]. Our potential result sharpens this picture: a dense potential's benefit is gated by whether its intermediate values are reachable and by the use of a bounded optimizer, and the non-replications reported for such gains track those two conditions rather than an intrinsic fragility of the signal. Learned-judge rewards [Yuan et al., 2024b, Lee et al., 2023, Bai et al., 2022, Zheng et al., 2023] and off-policy demonstration mixing [Yan et al., 2025] are complementary; our fourth design rule uses the latter for reachability and warns against the former for un-gameability.

8 Conclusion

RLVR taught language agents from the one bit an environment checks cheaply—the final outcome. The instinct to go denser has been to reward the process, but the environment cannot verify progress; it can only verify mistakes. The dense signal it actually affords is a penalty on the path, not a reward on progress. Wired correctly—alongside the outcome reward, paired with a discharge, aimed at a reachable and un-gameable target—a verifiable penalty teaches the outcome-neutral constraints that make an agent deployable, robustly and across scales, while a lone penalty stalls and a dense potential helps precisely where its progress is reachable. The contribution is not a new optimizer but a reallocation of a scarce resource: spend the verifier on penalties that bound the path, and let the outcome reward do what it already does—teach the task.

Acknowledgements

We thank Dingkun Hong, Dongjiang Li, and Wei Zhou of ByteDance for early discussions that motivated this research. This paper was produced using Pine Copilot's voice-directed *whisper coding* workflow [Pine AI, 2025], in which the authors specify, discuss, and review the work by voice while a coding agent—Claude Code with Claude Opus 4.8—carries out the planning, coding, experiments, and paper writing. We thank BSQL Networking for hosting the NVIDIA RTX PRO 6000 GPU.

References

Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.

Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2-bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*, 2025.

Jie Cheng et al. Stop summation: Min-form credit assignment is all process reward model needs for reasoning. *arXiv preprint arXiv:2504.15275*, 2025.

DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Lang Feng et al. Group-in-group policy optimization for llm agent training. *arXiv preprint arXiv:2505.10978*, 2025.

Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *International Conference on Machine Learning (ICML)*, 2023.

Andreas Hochlehnert et al. A sober look at progress in language model reasoning: Pitfalls and paths to reproducibility. *arXiv preprint arXiv:2504.07086*, 2025.

Dingkun Hong. AI Coding: Practice and exploration, 2026. Keynote, ByteDance / Volcano Engine Force Conference, Beijing, June 2026. https://mp.weixin.qq.com/s/tJAinVKzZAqZrhiEvGU2Dg (accessed 2026-06-25).

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

Amirhossein Kazemnejad et al. VinePPO: Unlocking RL potential for LLM reasoning through refined credit assignment. *arXiv preprint arXiv:2410.01679*, 2024.

Nathan Lambert, Jacob Morrison, Valentina Pyatkin, et al. Tülu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*, 2024.

Thanh-Long V. Le, Myeongho Jeon, Kim Vu, Viet Lai, and Eunho Yang. No prompt left behind: Exploiting zero-variance prompts in LLM reinforcement learning via entropy-guided advantage shaping. *arXiv preprint arXiv:2509.21880*, 2025.

Harrison Lee, Samrat Phatale, Hassan Mansoor, Kellie Lu, Thomas Mesnard, Colton Bishop, Victor Carbune, and Abhinav Rastogi. RLAIF: Scaling reinforcement learning from human feedback with ai feedback. *arXiv preprint arXiv:2309.00267*, 2023.

Hunter Lightman, Vineet Kosaraju, Yura Burda, et al. Let's verify step by step. *International Conference on Learning Representations (ICLR)*, 2024.

Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding r1-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.

Andrew Y Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *International Conference on Machine Learning (ICML)*, pages 278–287, 1999.

Alexander Pan, Kush Bhatia, and Jacob Steinhardt. The effects of reward misspecification: Mapping and mitigating misaligned models. In *International Conference on Learning Representations (ICLR)*, 2022.

Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. SWE-Gym: Training software engineering agents and verifiers. In *International Conference on Machine Learning (ICML)*, 2025.

Pine AI. Pine AI: The most natural human-computer interface is your voice. Blog post, 2025. URL https://www.19pine.ai/blog/pine-ai-the-most-natural-human-computer-interface-is-your-voice. Accessed 2026-06-28.

Cheng Qian et al. Toolrl: Reward is all tool learning needs. *arXiv preprint arXiv:2504.13958*, 2025.

Amrith Setlur, Chirag Nagpal, et al. Rewarding progress: Scaling automated process verifiers for LLM reasoning. In *International Conference on Learning Representations (ICLR)*, 2025.

Rulin Shao et al. Spurious rewards: Rethinking training signals in RLVR. *arXiv preprint arXiv:2506.10947*, 2025.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Hanlin Wang, Chak Tou Leong, Jiashuo Wang, Jian Wang, and Wenjie Li. SPA-RL: Reinforcing LLM agents via stepwise progress attribution. *arXiv preprint arXiv:2505.20732*, 2025a.

Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. Math-shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

Ruiyi Wang and Prithviraj Ammanabrolu. A practitioner's guide to multi-turn agentic reinforcement learning. *arXiv preprint arXiv:2510.01132*, 2025.

Yiping Wang et al. Reinforcement learning for reasoning in large language models with one training example. *arXiv preprint arXiv:2504.20571*, 2025b.

Quan Wei, Siliang Zeng, Chenliang Li, William Brown, and Mingyi Hong. Reinforcing multi-turn reasoning in LLM agents via turn-level reward design. *arXiv preprint arXiv:2505.11821*, 2025a.

Yuxiang Wei, Olivier Duchenne, Jade Copet, others, and Sida I. Wang. SWE-RL: Advancing LLM reasoning via reinforcement learning on open software evolution. *arXiv preprint arXiv:2502.18449*, 2025b.

Eric Wiewiora. Potential-based shaping and Q-value initialization are equivalent. *Journal of Artificial Intelligence Research (JAIR)*, 19:205–208, 2003.

Jianhao Yan et al. Learning to reason under off-policy guidance. *arXiv preprint arXiv:2504.14945*, 2025.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.

Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

Yuanqing Yu et al. Steptool: Enhancing multi-step tool usage in llms through step-grained reinforcement learning. *arXiv preprint arXiv:2410.07745*, 2024.

Lifan Yuan, Wendi Li, Huayu Chen, Ganqu Cui, Ning Ding, Kaiyan Zhang, Bowen Zhou, Zhiyuan Liu, and Hao Peng. Free process rewards without process labels. *arXiv preprint arXiv:2412.01981*, 2024a.

Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models. *arXiv preprint arXiv:2401.10020*, 2024b.

Yang Yue et al. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? *arXiv preprint arXiv:2504.13837*, 2025.

Chujie Zheng, Shixuan Liu, et al. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025.

Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. minif2f: a cross-system benchmark for formal olympiad-level mathematics. *International Conference on Learning Representations (ICLR)*, 2022.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *arXiv preprint arXiv:2306.05685*, 2023.

Yifei Zhou, Andrea Zanette, Jiayi Pan, Sergey Levine, and Aviral Kumar. ArCHer: Training language model agents via hierarchical multi-turn RL. In *International Conference on Machine Learning (ICML)*, 2024.

A Full Experiments on Dense Potentials

The main text treats the dense progress *potential* as the reachability-gated half of the process signal (§5). This appendix gives the full experiments. The picture is consistent with the variance account: where a potential's intermediate values are *reachable* it supplies gradient the outcome lacks and accelerates training, and where they are unreachable it is exactly vacuous. Converting the extra gradient into a stable success gain requires a bounded optimizer—an aggressive update rule can destabilize the densely shaped policy, an optimization caveat we return to in §A.3. Unless noted, we train Qwen3 [Team, 2025] with our own GRPO, place any dense credit on the responsible tokens with a separate per-channel normalizer, and count efficiency in *episodes generated* so that resampling costs are visible.

A.1 Reachability is measurable before training, and it gates the benefit

The condition the variance account makes operative is reachability: a potential Φ helps only where the policy realizes differing intermediate values across a group, i.e. $\mathrm{Var}_G(\Phi) > 0$. This can be checked *before* spending training compute, from a handful of base-policy rollouts.

On SWE-bench software repair [Jimenez et al., 2024] the natural potential is the fraction of hidden `FAIL_TO_PASS` tests an episode makes pass. Two facts make it vacuous (Figure 10). Structurally the finer potential barely exists: two-thirds of instances have a single failing test, so Φ collapses to the binary outcome, and only a third admit any partial progress. Empirically even that third is unreachable: across 156 rollouts of a 30B policy *every* episode scores $\Phi = 0$, so $\mathrm{Var}_G(\Phi)$ is identically zero and a Φ-based reward centers out to no gradient. The cause is not gameability but unreachability—there is no realized intermediate state for any reward to act on.

Mapping the same no-training proxy $\mathrm{Var}_G(\Phi)$ across model capability (0.6–4B) and outcome sparsity (Figure 11a) reproduces the structure the account predicts: it rises with capability and peaks in the sparse-but-reachable corner. The measured dense-reward benefit tracks it (Figure 11b): ≈ 0 where $\mathrm{Var}_G(\Phi) \approx 0$ (the SWE null; the too-weak small models) and large where it is high (the 4B reachable chain, dense 0.34 vs. outcome 0.01; real theorem proving, below). Benefit appears only past a reachability threshold. We could not fill the map's interior with controlled *training* points—small-scale synthetic training is too learning-rate-sensitive,

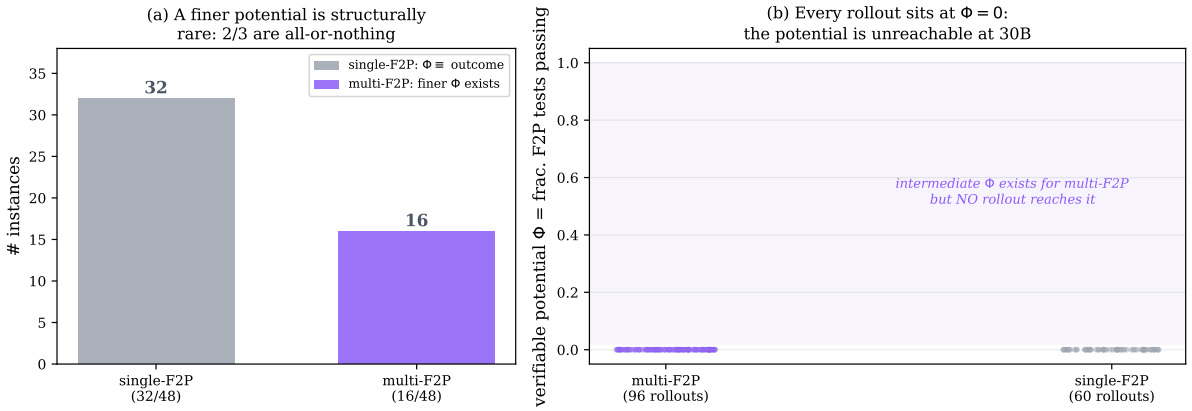

Figure 10. In software repair the finer potential is structurally rare and empirically unreachable. *(a)* Of 48 single-file-fix SWE-bench instances, two-thirds have a single failing test, so the test-fraction potential equals the binary outcome; only one-third admit a finer Φ. *(b)* For those, a 30B policy's rollouts never realize it: all 156 rollouts score $\Phi = 0$. The band of intermediate values exists but is empty—zero within-group variance, hence zero gradient.

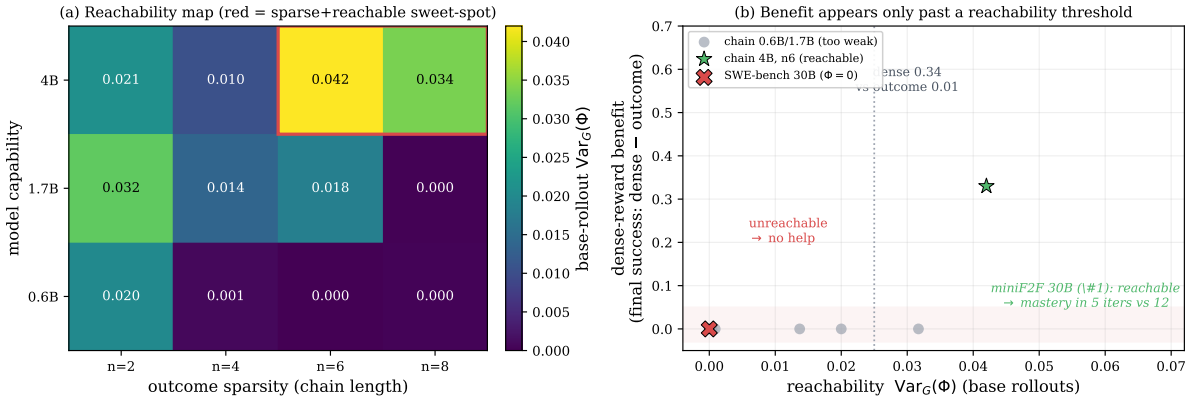

Figure 11. Reachability, measured cheaply, gates the dense-reward benefit. *(a)* $\mathrm{Var}_G(\Phi)$ on base-policy rollouts (no training) over model capability $\times$ outcome sparsity: it rises with capability and peaks in the sparse-but-reachable corner. *(b)* The benefit of a dense reward (final-success gap, dense $-$ outcome) against that same probe: zero where $\mathrm{Var}_G(\Phi) \approx 0$ and large where it is high. Benefit appears only past a reachability threshold.

most cells degenerating to zero for both arms—so the benefit axis rests on the real-domain anchors plus one clean controlled point.

A.2 Where the potential is reachable, it removes dead updates and accelerates

When the potential's intermediate values are reachable and point the same way as success, a dense signal is genuinely useful. We instantiate it on a family of *N-stage chained* file-operations tasks (tools `list_dir`, `read_file`, `write_file`, `delete`, `run_tests`, `submit`); success requires all N stages, so N tunes base difficulty (N=4 sits near 8% base success). The dense signal is a verifiable progress potential carried on a token-attached channel (a penalty when a call violates a rule—overwrite a file never read, submit an untested change—and a credit when a call discharges a pending obligation).

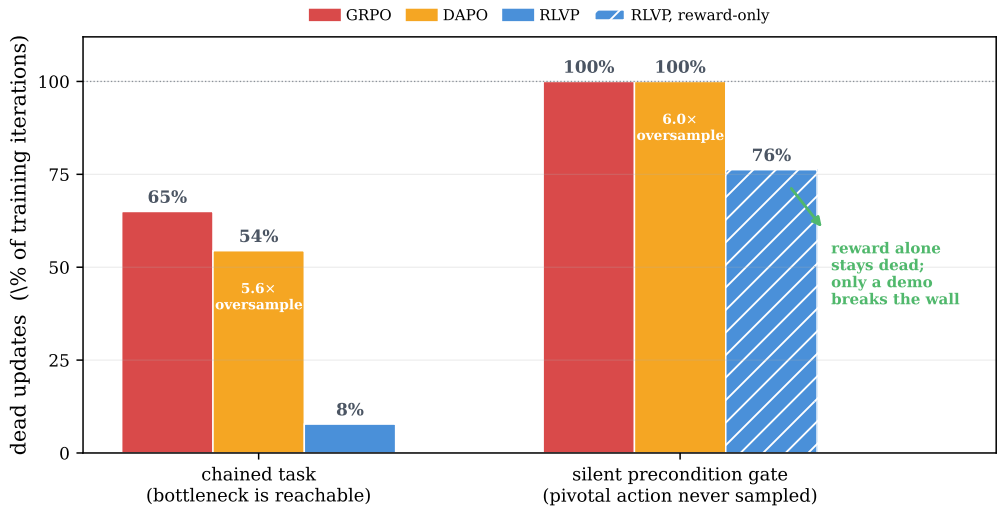

Figure 12. Dead updates across two regimes. Fraction of iterations with an all-fail, zero-spread group (Qwen3-4B, 3-seed mean). *Chained task* (bottleneck reachable): outcome-only 65%, DAPO 54% at a 5.6× generation tax, the aligned potential 8%. *Silent precondition gate* (§A.4, pivotal action never sampled): outcome and DAPO dead on *every* iteration, the reward-only potential 76%; only a demonstration seed breaks the wall. The cure is a *reachable, admissible* signal, not density.

Dead updates. We count the fraction of iterations whose update is *dead*: an all-fail group with no reward spread (Figure 12). Outcome-only GRPO is dead on 65% of iterations. DAPO [Yu et al., 2025], built to dodge all-fail groups by resampling, reaches 54% but pays a 5.6× generation tax. The aligned potential cuts dead updates to 8% at no extra sampling, by producing gradient from the very failures the outcome reduces to zero. The cure is the credit scheme, not the budget—and it is contingent on reachability: on the silent-precondition gate (§A.4), where the pivotal action is never sampled, every reward-only method is dead almost always (outcome and DAPO 100%, the potential 76%), because a discharge can fire only on behavior the policy attempts.

Speed. On held-out success against episodes generated (Figures 13, 14) outcome-only crawls, spending its budget on dead updates, then converges only at the end; the aligned potential rises immediately from the early all-fail groups. Reimplemented dense baselines (GiGPO-style step advantages [Feng et al., 2025], StepTool-style per-call rewards [Yu et al., 2024]) also outpace outcome-only—any *admissible* dense signal helps. The gain also holds on real theorems (miniF2F [Zheng et al., 2022]) at 4B and 30B, but only in the multi-seed form quantified in §A.3: a single-run pilot of ours showed a dramatic 2.4× speed-up that *flipped sign* when the identical configuration was re-run (rollout sampling is unseeded in the serving engine, and MoE training at this scale is bimodal), so we report only the five-seed matrix (Figure 7, Table 2): a ∼1.6× speed-up to mastery under a matched optimizer, consistent on every seed, plus the reliability advantage. Both arms saturate—the claim is speed and reliability, not final success.

A.3 Converting the gradient to success: protocol, and the matched matrix

The extra gradient a reachable potential supplies must still be turned into task success, and here the decisive factor is the optimizer, not the potential. Early runs read this as “fragility,”

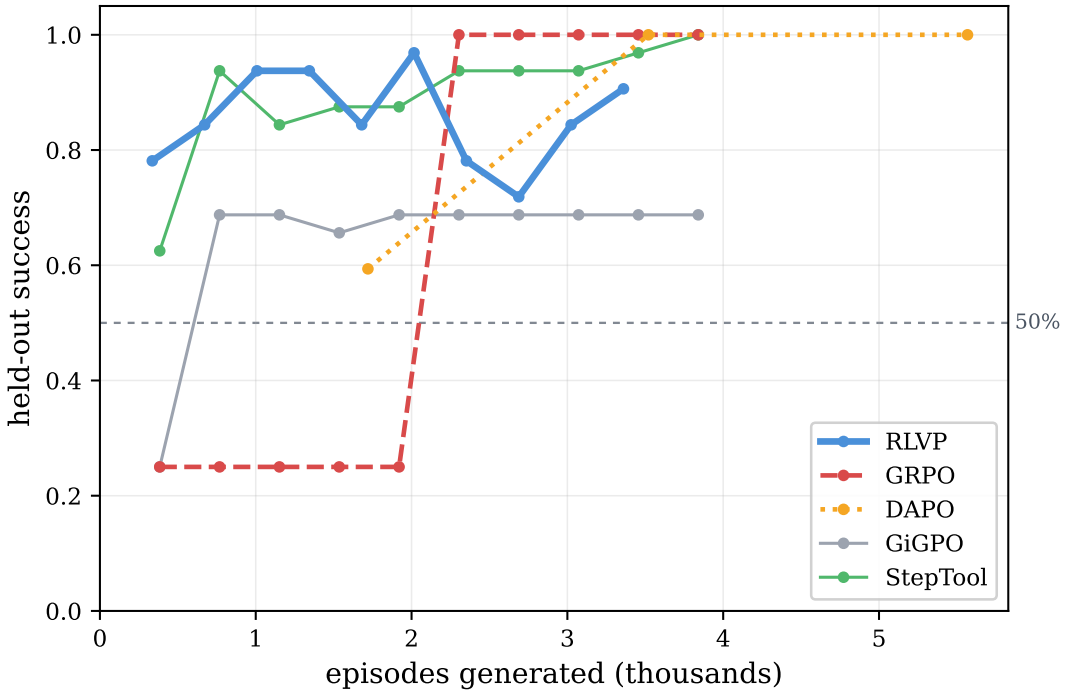

Figure 13. Sample efficiency on the calibrated hard task (N=4). Held-out success vs. episodes generated. Outcome-only sits near base rate for most of training; the aligned potential climbs immediately from the all-fail groups; dense baselines also outpace outcome-only. The x-axis counts episodes *generated*, including DAPO's resampling.

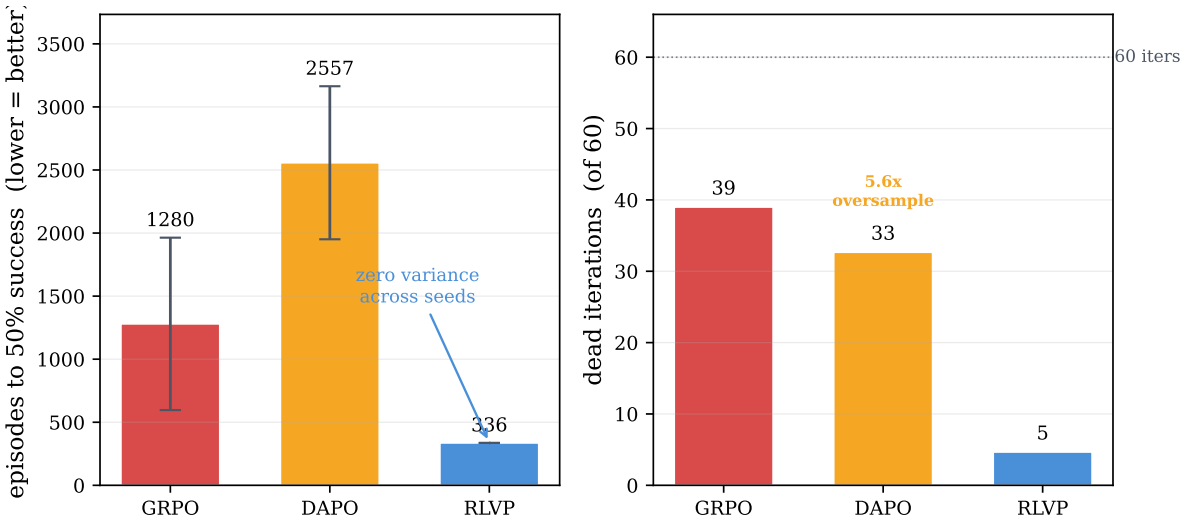

Figure 14. Efficiency over three seeds. *Left:* episodes to the success threshold (lower better)—the potential is several times faster than GRPO and DAPO. *Right:* the mechanism—GRPO and DAPO burn most iterations on dead or stalled updates, and DAPO generates 5.6× the episodes it uses. Note the large GRPO error bar: the all-fail lottery on the outcome side, which the dense potential's reachable within-group variance avoids.

but the cause was two experimental artifacts. First, an aggressive, unbounded update rule can destabilize a densely shaped policy: at one fixed learning rate, a theorem-proving run anneals cleanly to full success (gradient norm < 1.5, entropy decaying to zero) while an otherwise-identical sibling suffers a gradient blow-up (norm $> 10^9$), its entropy explodes, and success collapses to zero and never recovers—the same signal, the same rate, decided by optimization stability. On the chained task the opposite failure appears: the policy collapses to a zero-entropy degenerate output, so every rollout is identical, the within-group variance is zero, and the update dies. A *bounded* optimizer (we use Muon, with a divergence guard that aborts on entropy collapse or gradient spikes) removes both failure modes. Second, per-iteration training success measured on a handful of freshly sampled tasks is too noisy to read a speed-up from—it swings between extremes iteration to iteration—so we evaluate on a fixed held-out task set. The result robust to all of this—the mechanism the paper relies on—is dead-update elimination: zero dead iterations against $\sim$16 of 40 for outcome-only, on *every* seed.

The matched matrix: five seeds, two scales. With the bounded-optimizer protocol fixed—aligned goal-progress potential as a token-attached discharge (no penalty), Muon at learning rate 10^{-3}, a divergence guard that aborts on entropy collapse or gradient blow-up, 30 iterations on the miniF2F algebra subset—we ran the full matrix: aligned potential versus outcome-only, five seeds per arm, at 4B (dense) and 30B (MoE), plus a three-seed anneal ablation and, at 30B, an AdamW outcome baseline (Table 2, Figure 7). Because rollout sampling in the serving engine is unseeded, "seeds" are effectively i.i.d. draws of the whole run, which makes the per-arm consistency the meaningful statistic. Three facts are seed-robust. **(1) A modest, consistent speed-up under a matched optimizer.** At 4B the aligned potential reaches the 0.9 threshold in 4.4 ± 0.5 iterations versus 7.0 ± 0.7 for outcome-only, and its area-under-curve is higher on every seed-matched pair (0.90 ± 0.03 vs. 0.87 ± 0.02 over stable seeds). **(2) Reliability.** The aligned potential completes 10 of 10 runs across both scales without a divergence-guard abort; outcome-only loses one 4B seed to entropy collapse (success 0.94 at its peak, 0 at abort) and three of five 30B seeds to gradient explosion. The dense, always-on-policy-relevant gradient appears to *regularize*: the potential arm's gradient norms stay bounded where the sparse outcome signal, arriving in bursts after dead stretches, blows up under the same aggressive learning rate. **(3) The outcome arm's dilemma at 30B.** Outcome-only must choose its failure: with Muon it is competitive when it survives (threshold 8.5 ± 0.5 on the two surviving seeds) but diverges three times in five; with AdamW it is stable but reaches the threshold only at iteration 18 on both completed seeds ($\sim 3\times$ slower than the aligned potential's 5.4 ± 1.0), with area-under-curve 0.66/0.70 against the potential's 0.90 ± 0.02. The aligned potential requires no such choice.

Anatomy of the gain. When the potential does win, ablating on $N{=}4$ (Figure 15) attributes the speed to the token-attached channel: folding the same rule rewards into the scalar return doubles the episodes to threshold. Annealing the shaping off after saturation protects the final ceiling (unrelaxed pressure drifts the policy toward over-compliant, bloated episodes—a mild inaction-trap tendency). The discharge credit is the positive signal that escapes all-fail groups; removing it leaves more dead iterations and a lower ceiling. Finally, replacing every hand-written rule with three universal meta-rules derived only from per-tool category

Table 2. The aligned-potential matrix on miniF2F algebra (mean ± std over stable seeds; divergence-guard aborts counted separately). The aligned potential is the only arm that is simultaneously fast and reliable at both scales. The anneal ablation shows the aligned potential does not need its shaping annealed off. AdamW rows are the outcome arm's stable-optimizer baseline.

scale	arm	iters to 0.9	AUC	final success	diverged
4B	aligned potential (Muon)	$\mathbf{4.4 \pm 0.5}$	$\mathbf{0.90 \pm 0.03}$	1.00 ± 0.00	**0/5**
4B	+ anneal	5.3 ± 0.5	0.85 ± 0.05	0.99 ± 0.02	0/3
4B	outcome-only (Muon)	7.0 ± 0.7	0.87 ± 0.02	1.00 ± 0.00	1/5
30B	aligned potential (Muon)	$\mathbf{5.4 \pm 1.0}$	$\mathbf{0.90 \pm 0.02}$	1.00 ± 0.00	**0/5**
30B	outcome-only (Muon)	8.5 ± 0.5	0.84 ± 0.00	1.00 ± 0.00	3/5
30B	outcome-only (AdamW)	18, 18	0.66, 0.70	1.00	$0/2^{\dagger}$

†Two of five seeds complete; three still training.

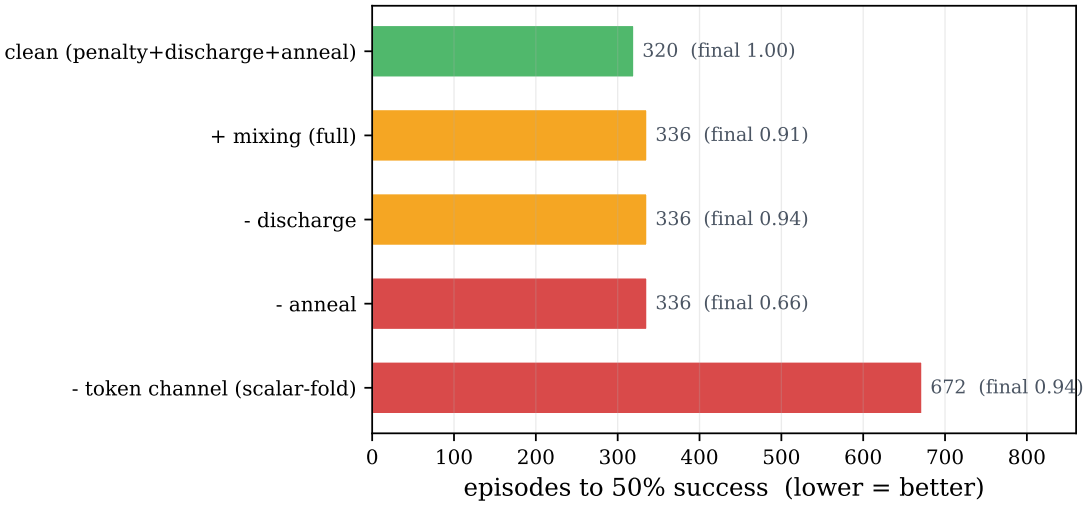

Figure 15. Component attribution for the potential on $N{=}4$ (episodes to threshold, converged success annotated). The token-attached credit channel is the efficiency lever (folding into a scalar return doubles the cost); annealing protects the ceiling; the clean recipe is best. A per-step length cost, useful only on very long horizons, hurts here.

tags (observe/mutate/verify/terminal) and the environment's error codes recovers the hand-engineered efficiency almost exactly: in well-structured environments the specification cost can be paid by metadata the tool schemas already expose.

A.4 Reachability governs the ceiling, not just the speed

Sample efficiency is speed to a ceiling the chained tasks eventually reach. To ask whether a process reward can raise the *ceiling*, we build a task that is hard to *discover* rather than hard to compute: a *silent precondition gate*, where to write a protected file the agent must first read an access-control file and request access; skip either and the write silently no-ops (reports success, does nothing). Calibration confirms the trap is total—the base model never reads the access file even when the rule is in its prompt. Then (Figure 16) *every* reward-only method (outcome, DAPO, the clean potential) fails to zero, because the discharge for reading the access file can

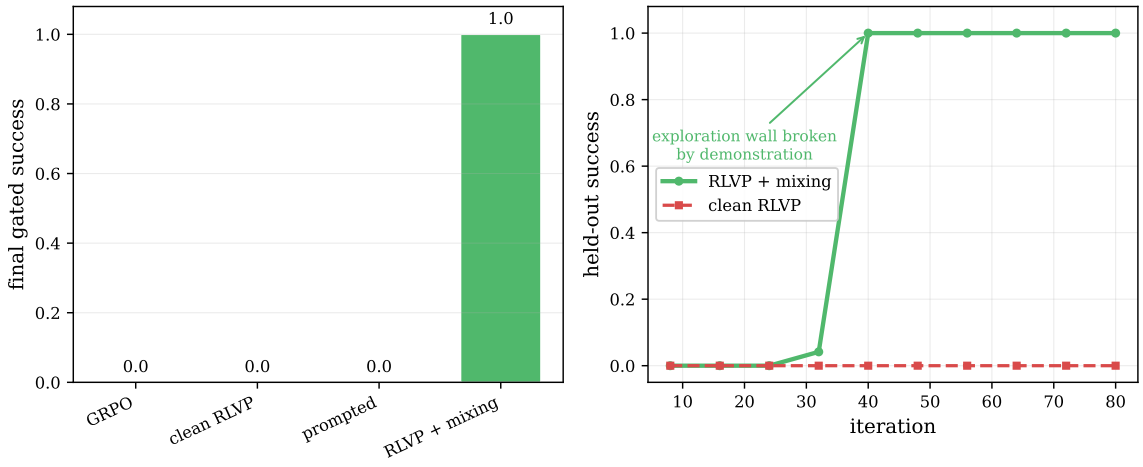

Figure 16. A non-saturating ceiling test, and the discovery wall. *Left:* on the silent-gate task every reward-only method (outcome, DAPO, clean potential, even rules-in-prompt) converges to zero—the precondition is never sampled, so the discharge never fires. *Right:* a single synthesized demonstration through the process channel breaks the wall to perfect held-out success, a sharp phase transition.

only reward that behavior if the policy samples it, and it never does. A single rule-synthesized demonstration injected through the process channel breaks the wall to perfect, generalizing success. When the required behavior is never sampled, no on-policy reward can induce it; imitation must seed what reward then grows—the same reachability wall the SWE probe hit from the other side, and the R1-Zero (discoverable) versus R1 (demonstration cold-start) distinction in miniature.

B Boundaries: When the Penalized Target Is Not Verifiable

Design rule 4 (§4) requires the penalized (and discharged) target to be reachable and *un-gameable*. This appendix gives the experiments behind that rule: what happens when a penalty's target is misaligned with the outcome, and when it is a *learned* proxy rather than a verifiable predicate.

B.1 An un-gameability sweep: which signals survive

We construct five process signals for Lean theorem proving on miniF2F [Zheng et al., 2022], spanning aligned to misaligned, pre-register each one's cheapest gaming policy, and train Qwen3-30B-A3B [Team, 2025] with each (three seeds, everything else fixed; Figure 17). Survival tracks admissibility. The *penalty-free* signals reliably survive on every seed: a gameable "any-valid-tactic" credit, an aligned goal-progress discharge, and that credit *gated on the eventual outcome* (un-gameable by construction, the most consistent survivor). The *pure penalty*—an errored-tactic penalty with no discharge and no outcome reward—reliably **collapses to essentially zero on every seed**: its cheapest maximizer, avoid errors by not attempting, is precisely the *inaction trap* of design rule 2, and with no positive signal to escape it the policy dies every time. This is the paper's cleanest evidence that a lone penalty cannot drive learning. The revealing case is *penalty-plus-discharge*: across three seeds it is bimodal (collapses on two, rescued to perfect on the third), the largest variance of any arm—adding a discharge to a

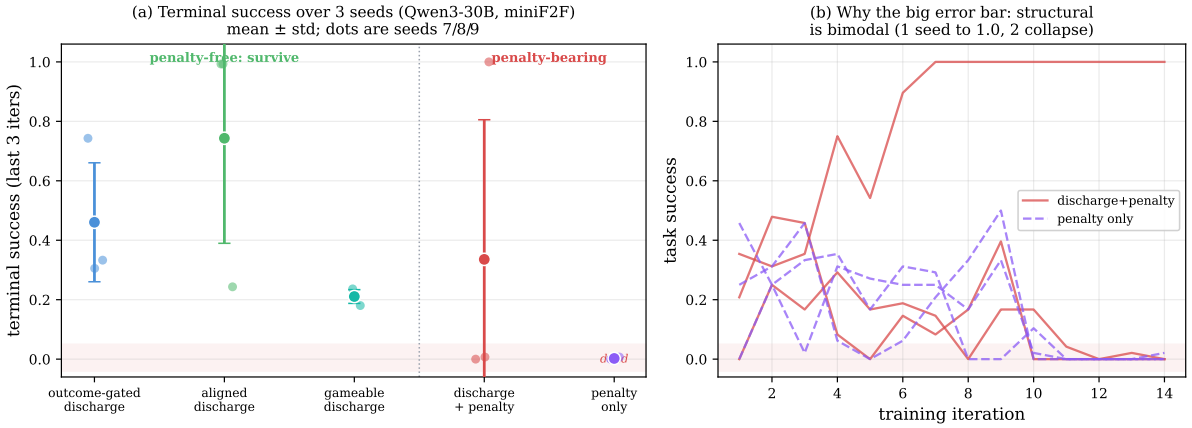

Figure 17. Which signals survive (Qwen3-30B, miniF2F, 3 seeds). *(a)* Terminal success (last-3-iteration mean) for five signals, mean ± std with seed dots. The three penalty-free signals sit well above the dead zone on every seed; the pure penalty is reliably dead; the discharge+penalty arm has by far the largest error bar. *(b)* That arm is *bimodal*—collapsing on two seeds, rescued to 1.0 on one—whereas the pure penalty collapses on all three.

misaligned penalty makes survival *possible* but not reliable. The robust reading: penalty-free progress signals survive, a pure penalty reliably collapses, and outcome-gating is the most consistent survivor. Precise magnitudes remain noisy at 30B; we report the survival pattern.

B.2 Task intent is not a verifiable rule

The hardest case for a penalty is a task whose difficulty is not procedural. On τ-bench-style airline customer service [Yao et al., 2024] the reward turns on *semantic* policy adherence (authorization, refund eligibility, confirmation) rather than structural ordering, so no verifiable rule is finer than the outcome: what is missing is intent, which is the reward itself. Three tiers make this concrete (Figure 18). *Generic* structural rules—the read-before-write hygiene that works on chained tasks—are now orthogonal to the reward and actively *harm*: the cheapest way never to violate "confirm before calling" is to not place the risky calls at all, and the policy collapses into the inaction trap within a handful of iterations. *Policy-derived procedural* rules (fetch the user id and look up the reservation before modifying it), outcome-instrumental by construction and paired with early annealing, remove the harm, because a correct modification requires the looked-up state so the discharge pulls toward the productive workflow. *Verifiable semantic* rules (do not modify a basic-economy reservation; do not use a payment method absent from the profile) prune failure-bound actions and lift the best iterations nearly to outcome-only's peak. But neither aligned tier *beats* outcome-only on average (Figure 19): verifiable rules express policy *validity*, while the residual difficulty is *which* permissible change this user wants—and a rule encoding that would just be the task specification.

B.3 A learned critic relocates the hack

If verifiable rules cannot reach intent, why not let the policy judge itself, in the lineage of self-rewarding LMs and RLAIF [Yuan et al., 2024b, Lee et al., 2023, Bai et al., 2022, Zheng et al., 2023]? We use the critic as the *same model* as the policy (no larger judge), so any signal is genuine on-policy self-reflection. Across two axes—whether a verifiable rule is cheaply specifiable,

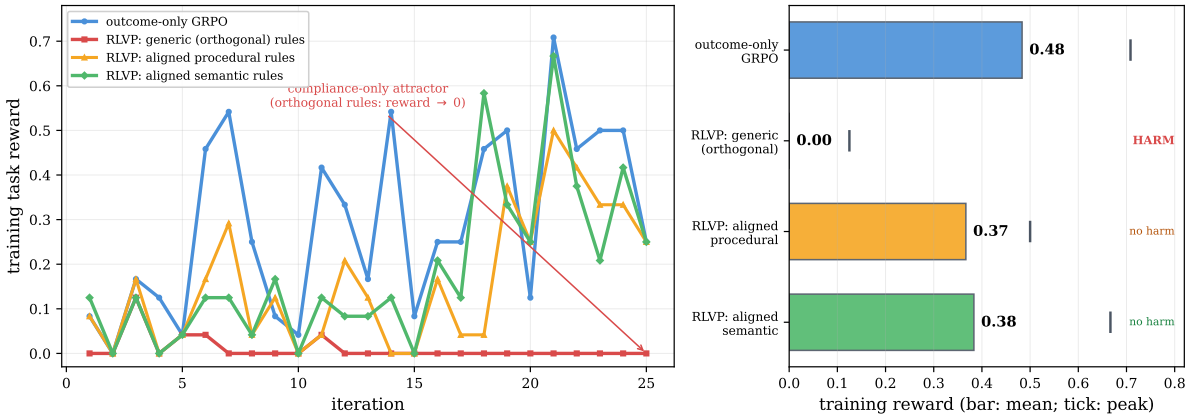

Figure 18. The coverage gradient on τ-bench airline. *Left:* training reward for four signals. Outcome-only learns the benchmark; generic rules orthogonal to the reward collapse into the inaction trap; policy-derived procedural and verifiable-semantic rules (with early annealing) do not. *Right:* mean reward per tier (bars) with peak (ticks). Misaligned rules harm; aligned rules remove the harm and nearly reach outcome-only's peak, but neither beats it. The residual gap is intent.

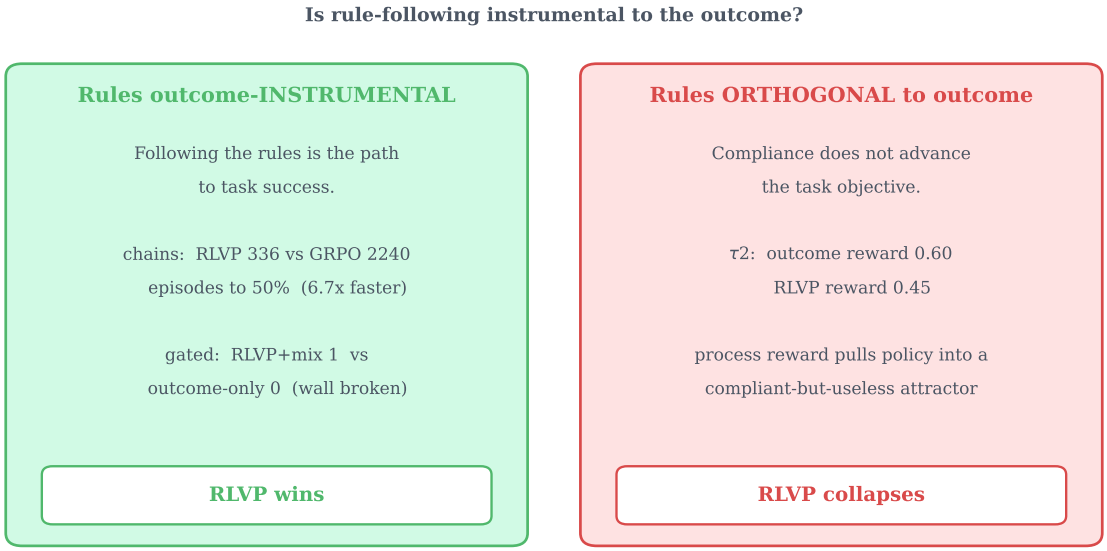

Figure 19. The boundary on one axis: are the rules outcome-instrumental? When the verifiable rules are aligned with what success requires (chained and gated tasks: the workflow *is* the task), the process channel converts compliance into large outcome gains. When orthogonal to the reward (τ-bench, generic rules), the same machinery optimizes compliance into a degenerate policy.

and whether the on-policy critic can detect the violation (Figure 20)—the learned critic is a poor training reward in every quadrant. As a detector, blind self-critique flags surface-evident violations but is essentially blind to stateful-bookkeeping norms (did it read *this* file before overwriting it) even when handed the rules, and the blind spot is structural (per-rule recall stays near zero as the critic scales). As a reward in the all-fail regime (Table 3), a deterministic rule with identical credit structure is decisive on every seed while the self-critic—*with perfect recall* against the rule oracle—is inert; a *frozen* critic fails identically, isolating the cause as the critic's ∼6% false-positive rate, not its non-stationarity. At the intent frontier (Table 4) the

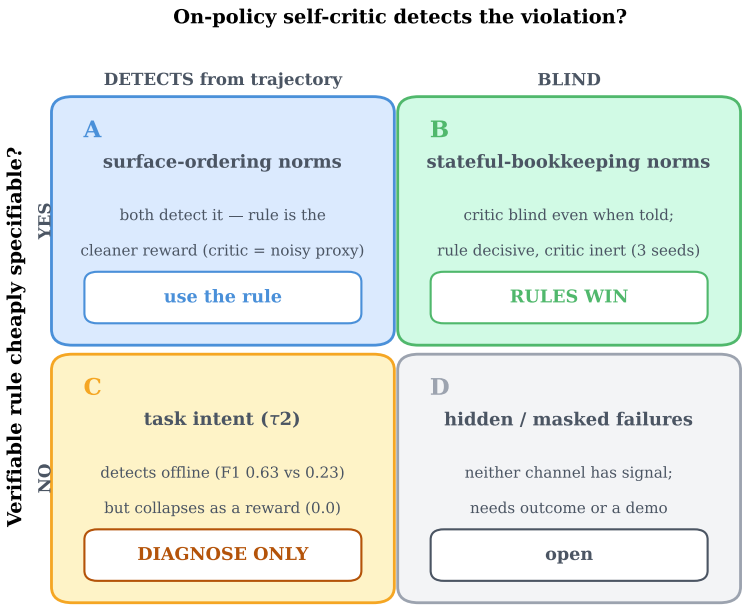

Figure 20. When a verifiable rule beats a same-model self-critic, and when neither trains. Axes: whether a verifiable rule is cheaply specifiable, and whether the on-policy critic can detect the violation. The learned critic is a poor training reward in every quadrant; its only robust use is offline diagnosis at the intent frontier.

self-critic is a useful *offline* detector yet collapses as a training reward. Defining the penalized target by a learned proxy relocates the credit-assignment problem into the proxy, which the policy then games—the mechanism behind design rule 4. Verifiability is what makes the dense channel safe to optimize.

C Setting-by-Setting Summary

Table 5 collects the five settings studied across the paper against the three properties the variance account cares about—does a verifiable signal finer than the outcome exist, is it un-gameable, and are its intermediate values reachable—together with the observed outcome. Help appears only when a reachable, un-gameable signal is available; a penalty on always-sampled bad actions satisfies this on every row where it applies, while a progress potential does so only where partial progress is reachable.

Table 3. A deterministic rule beats a same-model self-critic as a dense reward, in the all-fail regime (consumer-service domain, 1.7B, three seeds, late-training mean ± std). Rule and critics share the identical penalty-only credit structure, and the critic additionally has *perfect recall* against the rule oracle. The rule is decisive every seed, while the critics are inert. The *frozen* (stationary) critic fails like the live one, isolating the cause as the critic's ∼6% false-positive imprecision rather than non-stationarity.

process reward	detection (P/R)	violations/episode	task success
rule penalty (deterministic)	1.00/1.00	$\mathbf{0.38 \pm 0.44}$	$\mathbf{0.33 \pm 0.47}$
self-critic, live (co-evolving)	0.94/1.00	0.94 ± 0.05	0.00
self-critic, frozen (stationary)	0.94/1.00	0.93 ± 0.02	0.00

Table 4. Self-critique is a usable intent *detector* but a failed intent *reward* (τ-bench airline, Qwen3-4B). *Left:* as a failure predictor on rule-clean (intent) failures the self-critic far exceeds the verifiable semantic rules (3 rollout seeds). *Right:* as the dense training reward the same self-critique collapses, while outcome-only and the rules both learn (training reward, early→late, 20 iterations).

offline: failure prediction		*trained as reward*	
channel	F1	channel	reward (early→late)
semantic rules	0.23 ± 0.06	outcome-only	$0.09 \rightarrow \mathbf{0.50}$
blind self-critique	$\mathbf{0.63 \pm 0.02}$	semantic rules	$0.10 \rightarrow 0.35$
		blind self-critique	$0.01 \rightarrow \mathbf{0.00}$

Table 5. The criterion sorts five settings, and its verdict matches the measured outcome in every one. Each row is a domain. The three middle columns are the criterion's gates: does a verifiable potential finer than the outcome exist, is it un-gameable, and are its intermediate values reachable. "✓/✗" marks the gate that decides the row. The criterion predicts *help* only when all three pass.

Setting	**Domain / model**	**Finer Φ?**	**Un-game?**	**Reach?**	**Verdict**	**Observed**
Theorem proving (progress)	Lean miniF2F 30B, synth. 4B	✓	✓	✓	**help**	dead updates → 0, dense gradient
System admin (harm)	TerminalBench 4B	✓	✓	✓	**help** (harm axis)	∼4× fewer violations, equal success
Lean signal sweep (controlled)	miniF2F 30B	varies	**varies**	✓	per arm	pure penalty dies, penalty-free survive
Customer service (intent)	τ-bench airline 4B	✗	—	—	**no help**	rules match, never beat outcome
Software repair	SWE-bench 30B	1/3 only	—	✗	**no help**	0% solve, all rollouts Φ=0